Import libraries

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
seed = 42
```

Load Bank Personal Loan Modelling dataset & explore

- Dữ liệu được lấy từ Statlog (German Credit Data) Data Set https://archive.ics.uci.edu/ml/datasets/Statlog+%28German+Credit+Data%29
- mỗi người được biểu diễn bằng 24 thuộc tính
- Nhãn 1: là người đó có tín dụng tốt
- Nhãn 0: là người đó có tín dụng xấu
- Ma trận chi phí (cost matrix): Chi phí khi đánh giá sai tín dụng xấu thành tốt (5) cao hơn khi đánh giá sai tín dụng tốt thành xấu (1)

In [2]:
```
cost_matrix = np.array([[0, 5], [1, 0]])
print('Nếu tín dụng xấu được phân là xấu, chi phí:', cost_matrix[0][0])
print('Nếu tín dụng xấu được phân là tốt, chi phí:', cost_matrix[0][1])
print('Nếu tín dụng tốt được phân là xấu, chi phí:', cost_matrix[1][0])
print('Nếu tín dụng tốt được phân là tốt, chi phí:', cost_matrix[1][1])
```

```
Nếu tín dụng xấu được phân là xấu, chi phí: 0
Nếu tín dụng xấu được phân là tốt, chi phí: 5
Nếu tín dụng tốt được phân là xấu, chi phí: 1
Nếu tín dụng tốt được phân là tốt, chi phí: 0
```

In [3]:
```
data_array = np.genfromtxt('german.data-numeric')
data_frame = df = pd.DataFrame(data_array, columns=['A'+str(i) for i in range(1, 25
# ban đầu nhãn là 1: good, 2: bad
# để đơn giản ta chuyển về 1: good, 0: bad
data_frame['label'] = data_frame['label'].replace(2, 0)
data_frame
```

Out[3]:

	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10	...	A16	A17	A18	A19	A20	A21
0	1.0	6.0	4.0	12.0	5.0	5.0	3.0	4.0	1.0	67.0	...	0.0	0.0	1.0	0.0	0.0	1.0
1	2.0	48.0	2.0	60.0	1.0	3.0	2.0	2.0	1.0	22.0	...	0.0	0.0	1.0	0.0	0.0	1.0
2	4.0	12.0	4.0	21.0	1.0	4.0	3.0	3.0	1.0	49.0	...	0.0	0.0	1.0	0.0	0.0	1.0
3	1.0	42.0	2.0	79.0	1.0	4.0	3.0	4.0	2.0	45.0	...	0.0	0.0	0.0	0.0	0.0	0.0
4	1.0	24.0	3.0	49.0	1.0	3.0	3.0	4.0	4.0	53.0	...	1.0	0.0	1.0	0.0	0.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
995	4.0	12.0	2.0	17.0	1.0	4.0	2.0	4.0	1.0	31.0	...	0.0	0.0	1.0	0.0	0.0	1.0
996	1.0	30.0	2.0	39.0	1.0	3.0	1.0	4.0	2.0	40.0	...	0.0	1.0	1.0	0.0	0.0	1.0
997	4.0	12.0	2.0	8.0	1.0	5.0	3.0	4.0	3.0	38.0	...	0.0	0.0	1.0	0.0	0.0	1.0
998	1.0	45.0	2.0	18.0	1.0	3.0	3.0	4.0	4.0	23.0	...	0.0	0.0	1.0	0.0	0.0	0.0
999	2.0	45.0	4.0	46.0	2.0	1.0	3.0	4.0	3.0	27.0	...	0.0	1.0	1.0	0.0	0.0	1.0

1000 rows × 25 columns

In [4]:

```
data_frame.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 25 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   A1      1000 non-null   float64
 1   A2      1000 non-null   float64
 2   A3      1000 non-null   float64
 3   A4      1000 non-null   float64
 4   A5      1000 non-null   float64
 5   A6      1000 non-null   float64
 6   A7      1000 non-null   float64
 7   A8      1000 non-null   float64
 8   A9      1000 non-null   float64
 9   A10     1000 non-null   float64
 10  A11     1000 non-null   float64
 11  A12     1000 non-null   float64
 12  A13     1000 non-null   float64
 13  A14     1000 non-null   float64
 14  A15     1000 non-null   float64
 15  A16     1000 non-null   float64
 16  A17     1000 non-null   float64
 17  A18     1000 non-null   float64
 18  A19     1000 non-null   float64
 19  A20     1000 non-null   float64
 20  A21     1000 non-null   float64
 21  A22     1000 non-null   float64
 22  A23     1000 non-null   float64
 23  A24     1000 non-null   float64
 24  label   1000 non-null   float64
dtypes: float64(25)
memory usage: 195.4 KB
```

In [5]:
```
data_frame.describe()
```

Out[5]:

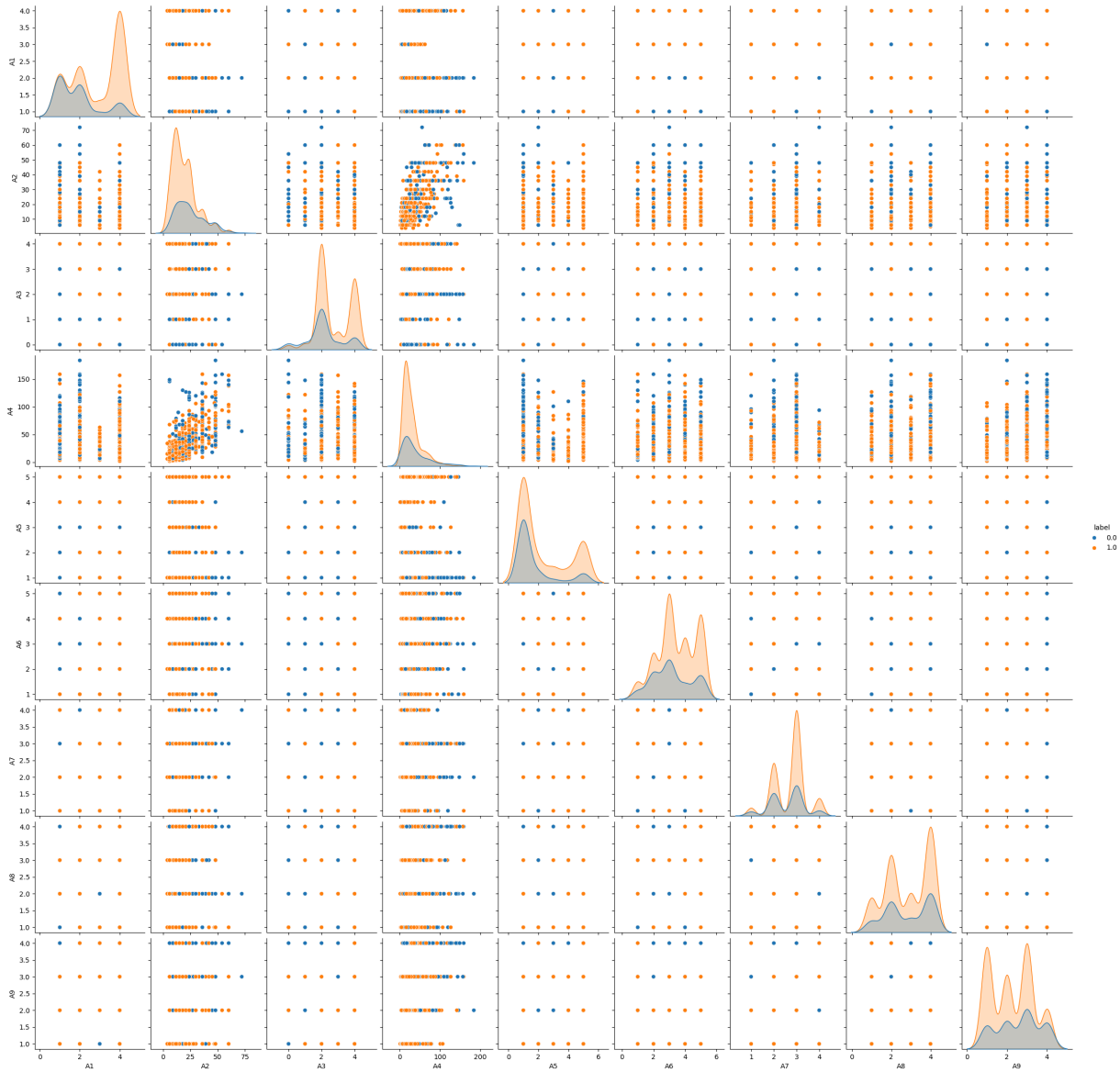
	A1	A2	A3	A4	A5	A6	
count	1000.000000	1000.000000	1000.00000	1000.000000	1000.000000	1000.000000	1000.0
mean	2.577000	20.903000	2.54500	32.711000	2.105000	3.384000	2.6
std	1.257638	12.058814	1.08312	28.252605	1.580023	1.208306	0.7
min	1.000000	4.000000	0.00000	2.000000	1.000000	1.000000	1.0
25%	1.000000	12.000000	2.00000	14.000000	1.000000	3.000000	2.0
50%	2.000000	18.000000	2.00000	23.000000	1.000000	3.000000	3.0
75%	4.000000	24.000000	4.00000	40.000000	3.000000	5.000000	3.0
max	4.000000	72.000000	4.00000	184.000000	5.000000	5.000000	4.0

8 rows × 25 columns

Data visualization

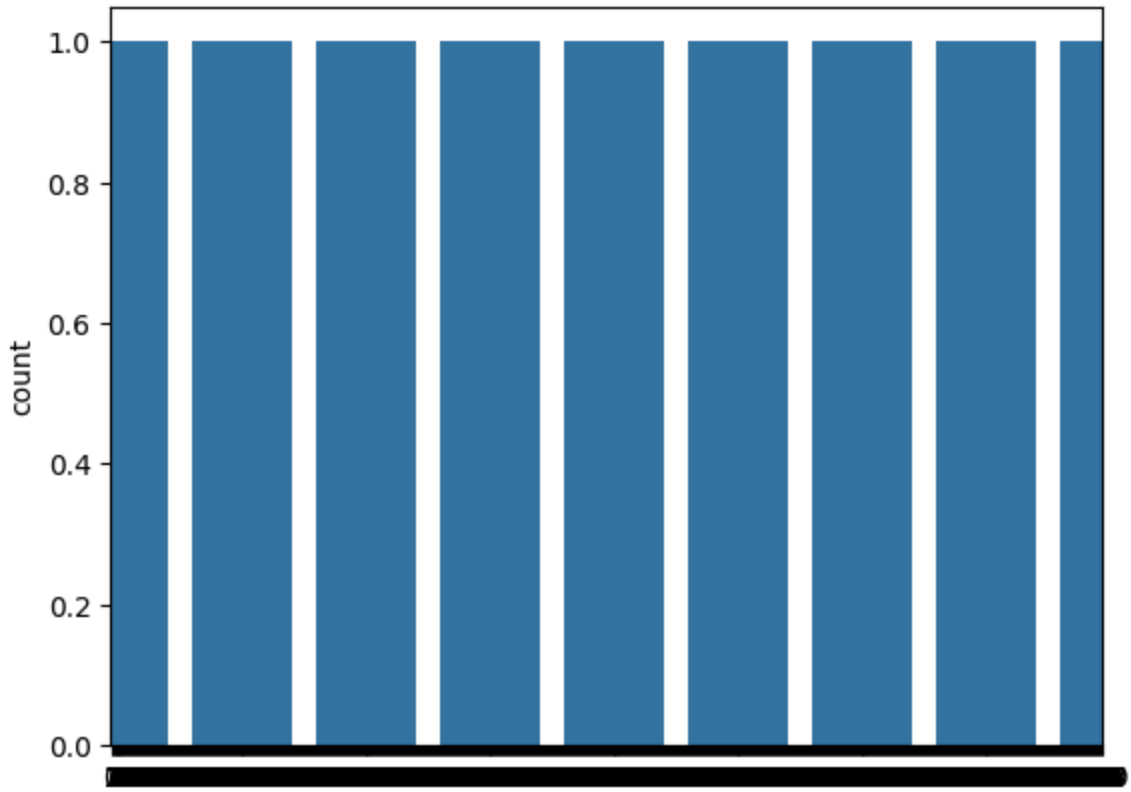
In [6]:
```
sns.pairplot(data_frame,
             hue='label',
             vars=['A'+str(i) for i in range(1, 10)])
```

Out[6]: <seaborn.axisgrid.PairGrid at 0x1f06c3ac050>



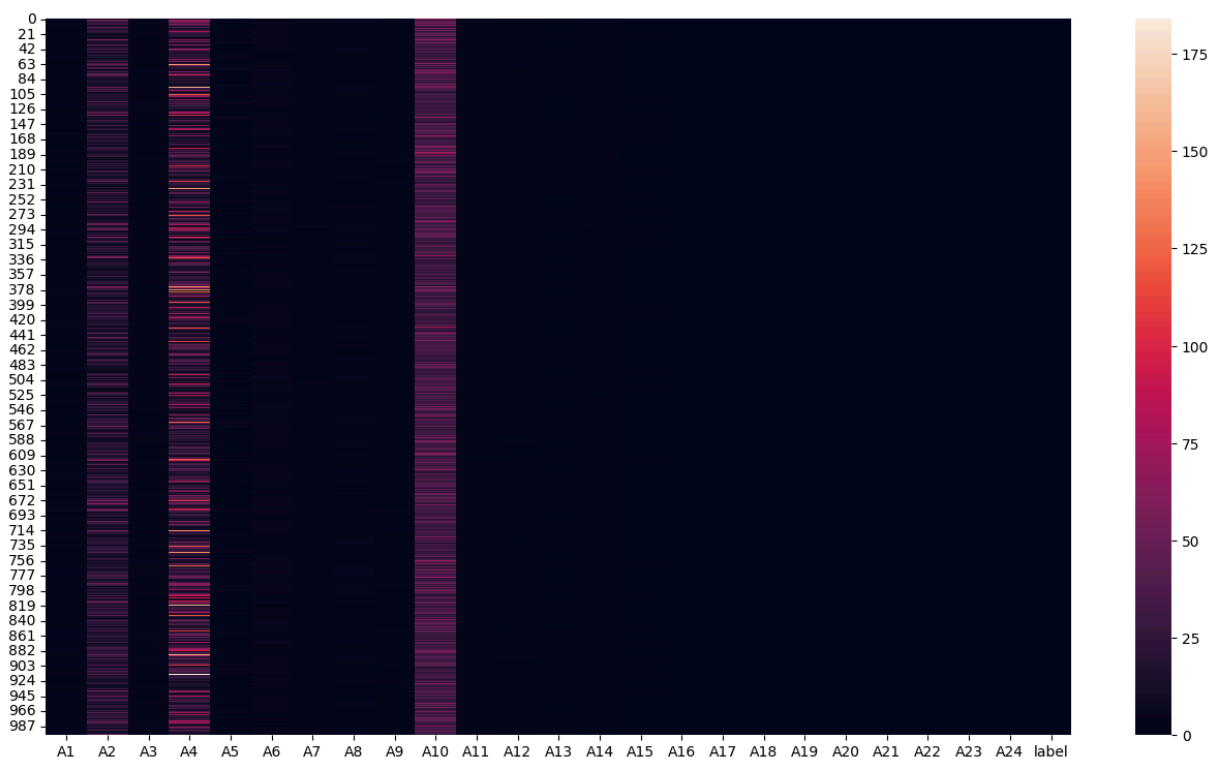
In [7]:
```
sns.countplot(data_frame['label'])
```

Out[7]: <Axes: ylabel='count'>



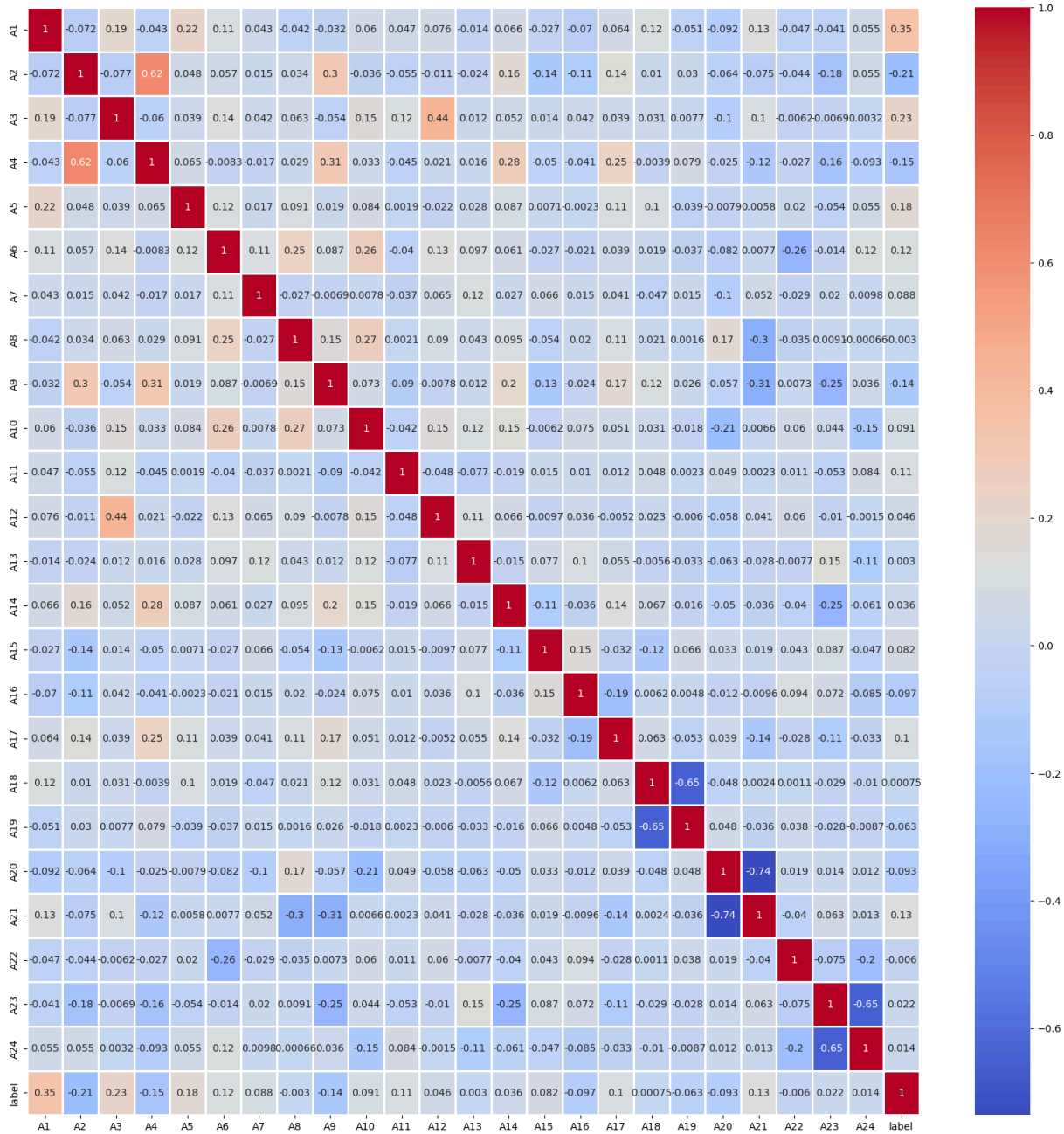
In [8]:
```
plt.figure(figsize=(16,9))
sns.heatmap(data_frame)
```

Out[8]: <Axes: >



In [9]:
```
plt.figure(figsize=(20,20))
sns.heatmap(data_frame.corr(), annot=True, cmap='coolwarm', linewidths=2)
```

Out[9]: <Axes: >



Data preprocessing

Normalization

In [10]:
```
X = data_frame.drop(['label'], axis=1)
# X = (X-X.mean())/X.var()
X = (X-X.min())/(X.max()-X.min())
X
```

Out[10]:

	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10
0	0.000000	0.029412	1.00	0.054945	1.00	1.00	0.666667	1.000000	0.000000	0.857143
1	0.333333	0.647059	0.50	0.318681	0.00	0.50	0.333333	0.333333	0.000000	0.053571
2	1.000000	0.117647	1.00	0.104396	0.00	0.75	0.666667	0.666667	0.000000	0.535714
3	0.000000	0.558824	0.50	0.423077	0.00	0.75	0.666667	1.000000	0.333333	0.464286
4	0.000000	0.294118	0.75	0.258242	0.00	0.50	0.666667	1.000000	1.000000	0.607143
...	...	...	...	...	...	...	...	...	...	...
995	1.000000	0.117647	0.50	0.082418	0.00	0.75	0.333333	1.000000	0.000000	0.214286
996	0.000000	0.382353	0.50	0.203297	0.00	0.50	0.000000	1.000000	0.333333	0.375000
997	1.000000	0.117647	0.50	0.032967	0.00	1.00	0.666667	1.000000	0.666667	0.339286
998	0.000000	0.602941	0.50	0.087912	0.00	0.50	0.666667	1.000000	1.000000	0.071429
999	0.333333	0.602941	1.00	0.241758	0.25	0.00	0.666667	1.000000	0.666667	0.142857

1000 rows × 24 columns

In [11]:
```
X.describe()
```

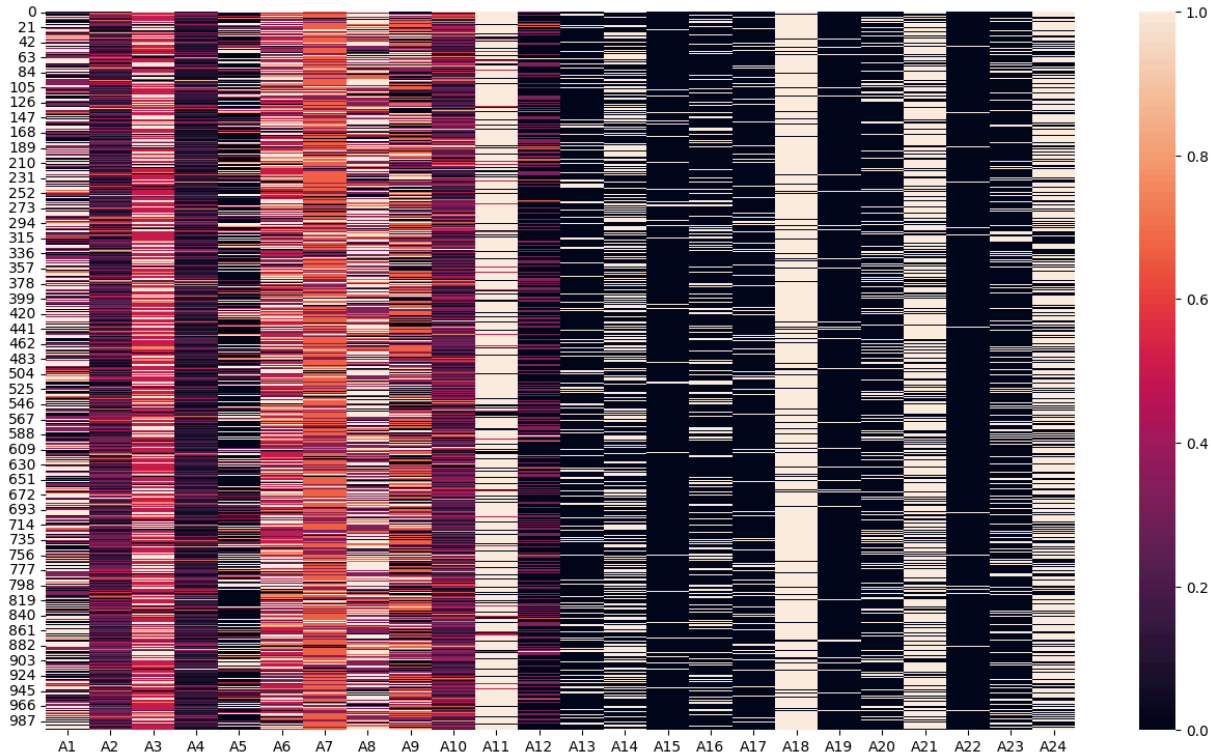
Out[11]:

	A1	A2	A3	A4	A5	A6	
count	1000.000000	1000.000000	1000.00000	1000.000000	1000.000000	1000.000000	1000.0
mean	0.525667	0.248574	0.63625	0.168742	0.276250	0.596000	0.5
std	0.419213	0.177336	0.27078	0.155234	0.395006	0.302077	0.2
min	0.000000	0.000000	0.00000	0.000000	0.000000	0.000000	0.0
25%	0.000000	0.117647	0.50000	0.065934	0.000000	0.500000	0.3
50%	0.333333	0.205882	0.50000	0.115385	0.000000	0.500000	0.6
75%	1.000000	0.294118	1.00000	0.208791	0.500000	1.000000	0.6
max	1.000000	1.000000	1.00000	1.000000	1.000000	1.000000	1.0

8 rows × 24 columns

In [12]:
```
plt.figure(figsize=(16,9))
sns.heatmap(X)
```

Out[12]:
```
<Axes: >
```



In [13]:
```
y = data_frame['label']
y
```

Out[13]:
```
0      1.0
1      0.0
2      1.0
3      1.0
4      0.0
      ...
995    1.0
996    1.0
997    1.0
998    0.0
999    1.0
Name: label, Length: 1000, dtype: float64
```

Chia dữ liệu làm 2 phần training và testing

- Training chiếm 80 % dữ liệu
- Testing chiếm 20 % dữ liệu

In [14]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
print("Dữ liệu training = ", X_train.shape, y_train.shape)
print("Dữ liệu testing = ", X_test.shape, y_test.shape)
```

```
Dữ liệu training =  (800, 24) (800,)
Dữ liệu testing =  (200, 24) (200,)
```

In [15]:
```
from sklearn.metrics import precision_score, recall_score, accuracy_score
from sklearn.neighbors import KNeighborsClassifier
```

```
import matplotlib.pyplot as plt

%matplotlib inline
```

K - Nearest Neightbor Classifier

Bài toán phân loại sử dụng KNN

Mục tiêu:

- Xây dựng được mô hình KNN sử dụng thư viện sklearn.
- Ứng dụng, hiểu cách áp dụng mô hình KNN vào giải quyết bài toán thực tế (vd: phân loại)
- Sử dụng độ đo Accuracy, Cost để làm độ đo đánh giá chất lượng mô hình.

Vấn đề:

- Có một tập các dữ liệu không có nhãn, làm sao để biết dữ liệu này là thuộc về nhãn nào.
- => Xây dựng mô hình học máy có thể phân loại.

Dữ liệu:

- Dữ liệu Bank Personal Loan Modelling
- Xem thêm: https://www.kaggle.com/teertha/personal-loan-modeling

Bài toán:

- Input: 1 mẫu dữ liệu $X = [x_1, x_2, \ldots x_n]$
- Output: nhãn y là 0 hoặc 1

In [16]:
```
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt
%matplotlib inline
```

Mô hình KNN

Sử dụng thư viện sklearn để xây dựng mô hình

- `KNeighborsClassifier(n_neighbors = 10, metric = 'minkowski', p = 2)`
- Số láng giềng: n_neighbors = 5
- Độ đo khoảng cách: Euclide p = 2

In [17]:
```
knn_classifier = KNeighborsClassifier(n_neighbors = 10, metric = 'minkowski', p = 2
knn_classifier.fit(X_train, y_train)
```

Out[17]:

```
▾ KNeighborsClassifier
▸ Parameters
```

Testing KNN model

Đánh giá theo các độ đo

In [18]:

```
def cost(y_true, y_pred):
    true_pos = ((y_true==y_pred)&(y_true==1.0))*0.0
    true_ne = ((y_true==y_pred)&(y_true==0.0))*0.0
    false_ne = ((y_true!=y_pred)&(y_true==1.0))*1.0
    false_pos = ((y_true!=y_pred)&(y_true==0.0))*5.0
    return sum(true_pos + true_ne + false_pos + false_ne)/len(y_true)
```

In [19]:

```
from sklearn.metrics import precision_score, recall_score, accuracy_score

print("Testing...\n")
y_pred_knn = knn_classifier.predict(X_test)
print('Accuracy: ', accuracy_score(y_test, y_pred_knn))
print('Cost: ', cost(y_test, y_pred_knn))
# print('Precision: ', precision_score(y_test, y_pred_knn))
# print('Recall: ', recall_score(y_test, y_pred_knn))
```

```
Testing...

Accuracy:  0.76
Cost:  0.86
```

Lựa chọn mô hình

Lựa chọn số lượng láng giềng

- Thay đổi số lượng láng riềng tìm giá trị cho kết quả phân loại tốt nhất

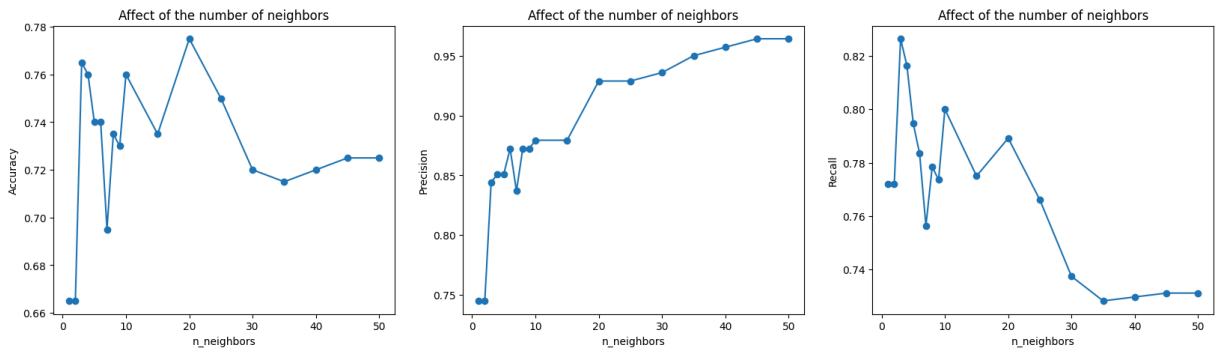
In [20]:

```
print("Evaluation of KNN with different K values")
n_neighbors = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 15, 20, 25, 30, 35, 40, 45, 50]
models = []
acc, pre, re = [], [], []
for n in n_neighbors:
    knn_classifier = KNeighborsClassifier(n_neighbors = n, metric = 'minkowski', p
    knn_classifier.fit(X_train, y_train)
    y_pred = knn_classifier.predict(X_test)
    acc.append(accuracy_score(y_pred, y_test))
    pre.append(precision_score(y_pred, y_test))
    re.append(recall_score(y_pred, y_test))

fig, axs = plt.subplots(1, 3, figsize = (20, 5))
```

```
axs[0].plot(n_neighbors, acc, marker = "o")
axs[0].set_xlabel("n_neighbors")
axs[0].set_ylabel("Accuracy")
axs[0].set_title("Affect of the number of neighbors")

axs[1].plot(n_neighbors, pre, marker = "o")
axs[1].set_xlabel("n_neighbors")
axs[1].set_ylabel("Precision")
axs[1].set_title("Affect of the number of neighbors")

axs[2].plot(n_neighbors, re, marker = "o")
axs[2].set_xlabel("n_neighbors")
axs[2].set_ylabel("Recall")
axs[2].set_title("Affect of the number of neighbors")
```

Evaluation of KNN with different K values

Out[20]: Text(0.5, 1.0, 'Affect of the number of neighbors')



Lựa chọn thuộc tính

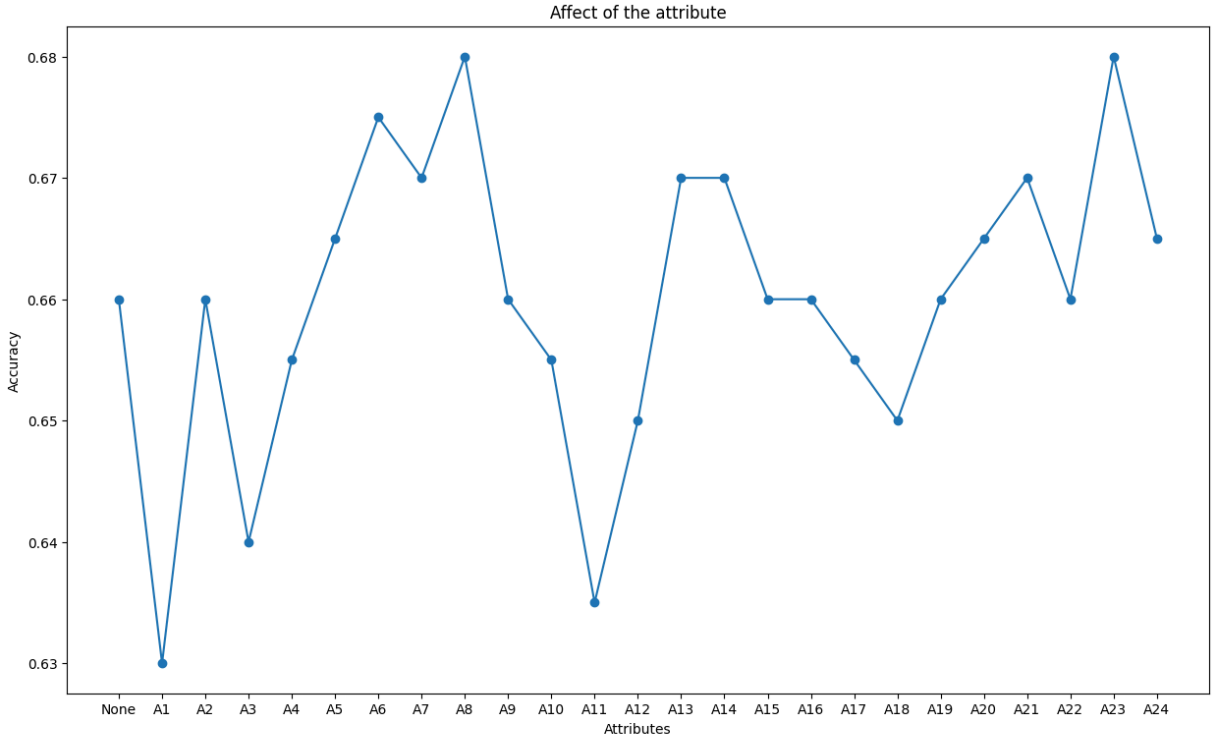
- Các thuộc tính: A1-24
- Thử loại bỏ từng thuộc tính ra khỏi dữ liệu xem chúng ảnh hưởng như thế nào tới kết quả phân loại.
- Các thuộc tính nào nên được sử dụng để cho kết quả phân loại tốt nhất ?

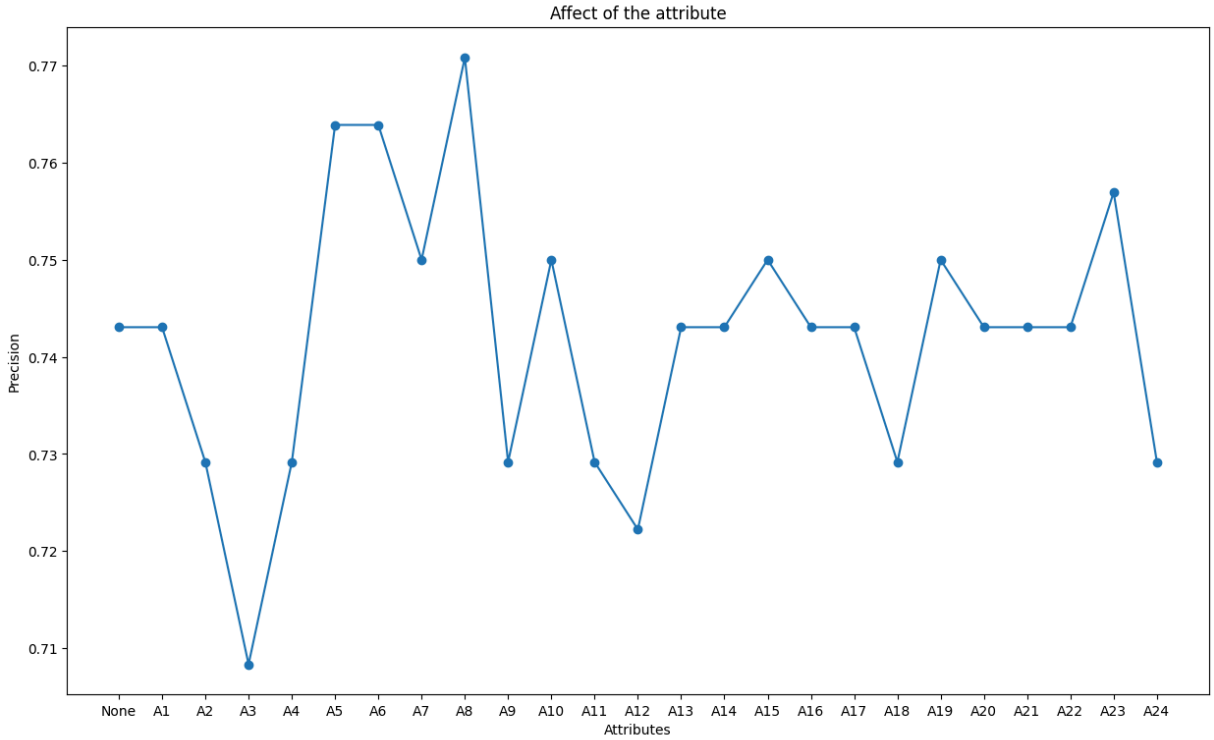
In [22]:
```
features = data_frame.drop(['label'], axis=1).columns.tolist()
features.insert(0,"None")
acc_f, pre_f, re_f = [], [], []
for feature in features:
    if feature == "None":
        X = data_frame.drop(['label'], axis=1)
    else:
        X = data_frame.drop(feature, axis=1).drop(['label'], axis=1)
    X = (X-X.min())/(X.max()-X.min())
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random
    knn_classifier = KNeighborsClassifier(n_neighbors = 1, metric = 'minkowski', p
    knn_classifier.fit(X_train, y_train)
    y_pred = knn_classifier.predict(X_test)
    acc_f.append(accuracy_score(y_pred, y_test))
    pre_f.append(precision_score(y_pred, y_test))
    re_f.append(recall_score(y_pred, y_test))

fig, axs = plt.subplots(3, 1, figsize = (15, 30))
```

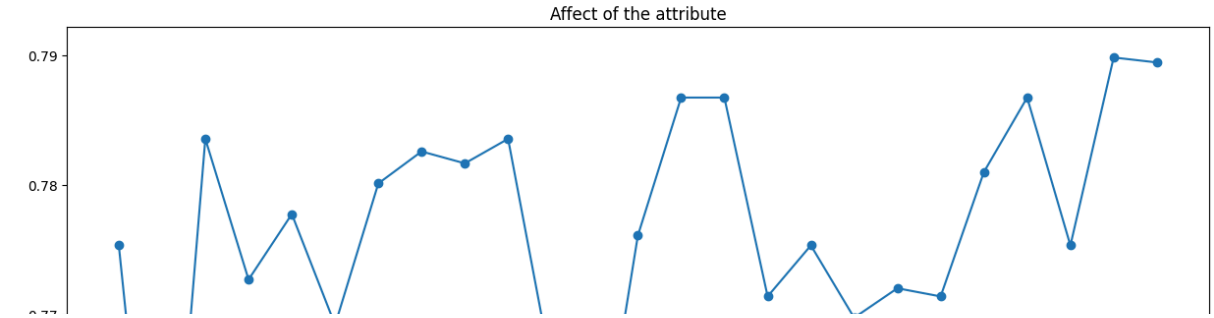
```python
axs[0].plot(features, acc_f, marker = "o")
axs[0].set_xlabel("Attributes")
axs[0].set_ylabel("Accuracy")
axs[0].set_title("Affect of the attribute")

axs[1].plot(features, pre_f, marker = "o")
axs[1].set_xlabel("Attributes")
axs[1].set_ylabel("Precision")
axs[1].set_title("Affect of the attribute")

axs[2].plot(features, re_f, marker = "o")
axs[2].set_xlabel("Attributes")
axs[2].set_ylabel("Recall")
axs[2].set_title("Affect of the attribute")
```

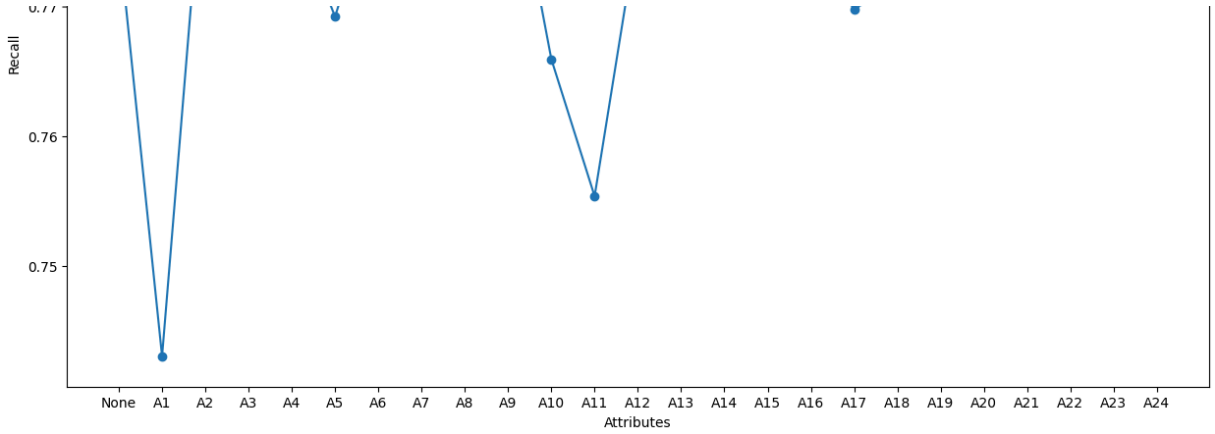
Out[22]:

```
Text(0.5, 1.0, 'Affect of the attribute')
```


Affect of the attribute
Accuracy
0.68
0.67
0.66
0.65
0.64
0.63
None A1 A2 A3 A4 A5 A6 A7 A8 A9 A10 A11 A12 A13 A14 A15 A16 A17 A18 A19 A20 A21 A22 A23 A24
Attributes

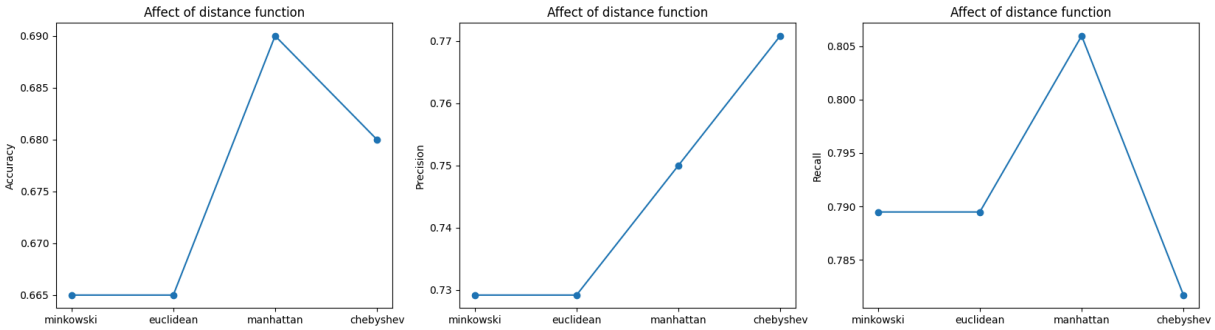

Affect of the attribute
Precision
0.77
0.76
0.75
0.74
0.73
0.72
0.71
None A1 A2 A3 A4 A5 A6 A7 A8 A9 A10 A11 A12 A13 A14 A15 A16 A17 A18 A19 A20 A21 A22 A23 A24
Attributes


Affect of the attribute
0.79
0.78



Lựa chọn hàm tính khoảng cách

- Hàm tính khoảng cách: minkowski, manhattan, euclidean, chebyshev
- Hàm tính khoảng cách nào là tốt nhất cho bài toán này ?

In [23]:
```
metrics = ['minkowski', 'euclidean', 'manhattan', 'chebyshev']
acc_metric, pre_metric, re_metric = [], [], []
for metric in metrics:
    knn_classifier = KNeighborsClassifier(n_neighbors = 1, metric = metric, p = 2,
    knn_classifier.fit(X_train, y_train)
    y_pred = knn_classifier.predict(X_test)
    acc_metric.append(accuracy_score(y_pred, y_test))
    pre_metric.append(precision_score(y_pred,y_test))
    re_metric.append(recall_score(y_pred, y_test))

fig, axs = plt.subplots(1, 3, figsize = (20, 5))

axs[0].plot(metrics, acc_metric, marker = "o")
axs[0].set_ylabel("Accuracy")
axs[0].set_title("Affect of distance function")

axs[1].plot(metrics, pre_metric, marker = "o")
axs[1].set_ylabel("Precision")
axs[1].set_title("Affect of distance function")

axs[2].plot(metrics, re_metric, marker = "o")
axs[2].set_ylabel("Recall")
axs[2].set_title("Affect of distance function")
```

Out[23]: Text(0.5, 1.0, 'Affect of distance function')